

Exercise 1: Employee Management System - Overview and Setup

Business Scenario:

You are developing an employee management system that will manage employee data, departments, and their relationships.

Instructions:

1. Creating a Spring Boot Project:

- Initialize a new Spring Boot project named **EmployeeManagementSystem**.
- Add dependencies: **Spring Data JPA, H2 Database, Spring Web, Lombok**.

2. Configuring Application Properties:

- Configure **application.properties** for H2 database connection.

spring.datasource.url=jdbc:h2:mem:testdb

spring.datasource.driverClassName=org.h2.Driver

spring.datasource.username=sa

spring.datasource.password=password

spring.jpa.database-platform=org.hibernate.dialect.H2Dialect

src/main/resources/application.properties

package com.cognizant.ems;

import org.springframework.boot.SpringApplication;

import org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication

public class EmployeeManagementSystemApplication {

public static void main(String[] args) {

SpringApplication.run(EmployeeManagementSystemApplication.class, args);

System.out.println("Employee Management System Started Successfully");

}

}

Exercise 2: Employee Management System - Creating Entities

Business Scenario:

Define JPA entities for Employee and Department with appropriate relationships.

Instructions:

1. Creating JPA Entities:

- Define **Employee** entity with fields: **id**, **name**, **email**, **department**.
- Define **Department** entity with fields: **id**, **name**.

2. Mapping Entities to Database Tables:

- Use annotations like **@Entity**, **@Table**, **@Id**, **@GeneratedValue**, etc.
- Define one-to-many relationship between **Department** and **Employee**.

```
package com.cognizant.ems.entity;

import java.util.List;

import jakarta.persistence.CascadeType;
import jakarta.persistence.Entity;
import jakarta.persistence.GeneratedValue;
import jakarta.persistence.GenerationType;
import jakarta.persistence.Id;
import jakarta.persistence.OneToOne;
import jakarta.persistence.Table;
import lombok.Data;

@Entity
@Table(name = "department")
@Data
public class Department {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
```

```
private Long id;

private String name;

@OneToMany(mappedBy = "department", cascade = CascadeType.ALL)

private List<Employee> employees;
}

package com.cognizant.ems.entity;

import jakarta.persistence.Entity;

import jakarta.persistence.FetchType;

import jakarta.persistence.GeneratedValue;

import jakarta.persistence.GenerationType;

import jakarta.persistence.Id;

import jakarta.persistence.JoinColumn;

import jakarta.persistence.ManyToOne;

import jakarta.persistence.Table;

import lombok.Data;

@Entity

@Table(name = "employee")

@Data

public class Employee {

    @Id

    @GeneratedValue(strategy = GenerationType.IDENTITY)

    private Long id;


    private String name;

    private String email;
```

```
@ManyToOne(fetch = FetchType.LAZY)
@JoinColumn(name = "department_id")
private Department department;
}
```

Exercise 3: Employee Management System - Creating Repositories

Business Scenario:

Create repositories for Employee and Department entities to perform CRUD operations.

Instructions:

1. Overview of Spring Data Repositories:

- Learn the benefits of using Spring Data repositories.

2. Creating Repositories:

- Create **EmployeeRepository** and **DepartmentRepository** interfaces extending **JpaRepository**.
- Define derived query methods in these repositories.

```
package com.cognizant.ems.repository;

import java.util.List;

import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.stereotype.Repository;

import com.cognizant.ems.entity.Employee;

@Repository
public interface EmployeeRepository extends JpaRepository<Employee, Long> {

    List<Employee> findByName(String name);

    Employee findByEmail(String email);

    List<Employee> findByDepartmentId(Long departmentId);}

package com.cognizant.ems.repository;
```

```
import java.util.List;

import org.springframework.data.jpa.repository.JpaRepository;

import org.springframework.stereotype.Repository;

import com.cognizant.ems.entity.Department;

@Repository

public interface DepartmentRepository extends JpaRepository<Department, Long> {

    Department findByName(String name);

    List<Department> findByNameContaining(String keyword);}


```

Exercise 4: Employee Management System - Implementing CRUD Operations

Business Scenario:

Implement CRUD operations for managing employees and departments.

Instructions:

1. Basic CRUD Operations:

- Use **JpaRepository** methods to create, read, update, and delete employees and departments.
- Implement RESTful endpoints for these operations using **EmployeeController** and **DepartmentController**.

```
package com.cognizant.ems.service;

import java.util.List;

import org.springframework.beans.factory.annotation.Autowired;

import org.springframework.stereotype.Service;

import com.cognizant.ems.entity.Employee;

import com.cognizant.ems.repository.EmployeeRepository;

@Service


```

```
public class EmployeeService {

    @Autowired

    private EmployeeRepository employeeRepository;

    public Employee saveEmployee(Employee employee) {

        return employeeRepository.save(employee);

    }

    public List<Employee> getAllEmployees() {

        return employeeRepository.findAll();

    }

    public Employee getEmployeeById(Long id) {

        return employeeRepository.findById(id).orElse(null);

    }

    public Employee updateEmployee(Employee employee) {

        return employeeRepository.save(employee);}

    public void deleteEmployee(Long id) {

        employeeRepository.deleteById(id);

    }

}

package com.cognizant.ems.service;

import java.util.List;

import org.springframework.beans.factory.annotation.Autowired;

import org.springframework.stereotype.Service;

import com.cognizant.ems.entity.Department;

import com.cognizant.ems.repository.DepartmentRepository;

public class DepartmentService {
```

```

private DepartmentRepository departmentRepository;

public Department saveDepartment(Department department) {
    return departmentRepository.save(department);
}

public List<Department> getAllDepartments() {
    return departmentRepository.findAll();
}

public Department getDepartmentById(Long id) {
    return departmentRepository.findById(id).orElse(null);
}

public Department updateDepartment(Department department) {
    return departmentRepository.save(department);
}

public void deleteDepartment(Long id) {
    departmentRepository.deleteById(id);
}
}

```

Exercise 5: Employee Management System - Defining Query Methods

Business Scenario:

Enhance your repository to support custom queries.

Instructions:

1. Defining Query Methods:

- Use keywords in method names to create custom query methods.
- Implement custom query methods using the **@Query** annotation.

2. Named Queries:

- Define and execute named queries with **@NamedQuery** and **@NamedQueries**.

```

package com.cognizant.ems.repository;

import java.util.List;

import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.data.jpa.repository.Query;
import org.springframework.stereotype.Repository;

import com.cognizant.ems.entity.Employee;

@Repository

public interface EmployeeRepository extends JpaRepository<Employee, Long> {

    List<Employee> findByName(String name);

    Employee findByEmail(String email);

    List<Employee> findByDepartmentId(Long departmentId);

    List<Employee> findByNameContaining(String keyword);

    @Query("SELECT e FROM Employee e WHERE e.email=?1")
    Employee getEmployeeByEmail(String email);

    @Query("SELECT e FROM Employee e WHERE e.department.id=?1")
    List<Employee> getEmployeesByDepartment(Long departmentId);

    @Query(name="Employee.findByEmployeeName")
    List<Employee> searchEmployeeByName(String name);
}

@NamedQueries({

    @NamedQuery(

        name="Employee.findByEmployeeName",

        query="SELECT e FROM Employee e WHERE e.name=:name"

    ),

    @NamedQuery(

```



```
        name="Employee.findAllEmployees",
        query="SELECT e FROM Employee e"
    )
})

@Entity
@Table(name="employee")
public class Employee {

    @Id
    @GeneratedValue(strategy=GenerationType.IDENTITY)
    private Long id;

    private String name;

    private String email;

    @ManyToOne
    @JoinColumn(name="department_id")
    private Department department;
}

package com.cognizant.ems.service;

import java.util.List;

import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Service;

import com.cognizant.ems.entity.Employee;
import com.cognizant.ems.repository.EmployeeRepository;

@Service
public class EmployeeService {

    @Autowired
```

```

        private EmployeeRepository employeeRepository;

        public List<Employee> getEmployeeByName(String name) {

            return employeeRepository.searchEmployeeByName(name);

        }
    }

package com.cognizant.ems.controller;

import java.util.List;

import org.springframework.beans.factory.annotation.Autowired;

import org.springframework.web.bind.annotation.*;

import com.cognizant.ems.entity.Employee;
import com.cognizant.ems.service.EmployeeService;

@RestController

@RequestMapping("/employees")

public class EmployeeController {

    @Autowired

    private EmployeeService employeeService;

    @GetMapping("/name/{name}")

    public List<Employee> getEmployeeByName(@PathVariable String name) {

        return employeeService.getEmployeeByName(name);

    }

}

```

Exercise 6: Employee Management System - Implementing Pagination and Sorting

Business Scenario:

Add pagination and sorting capabilities to your employee search functionality.

Instructions:

1. Pagination:

- Implement pagination for the employee list using **Page** and **Pageable**.

2. Sorting:

- Add sorting functionality to your queries.
- Combine pagination and sorting in your search endpoint.

```
package com.cognizant.ems.repository;

import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.stereotype.Repository;

import com.cognizant.ems.entity.Employee;

@Repository
public interface EmployeeRepository extends JpaRepository<Employee, Long> {
}

package com.cognizant.ems.service;

import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.data.domain.Page;
import org.springframework.data.domain.Pageable;
import org.springframework.stereotype.Service;

import com.cognizant.ems.entity.Employee;
import com.cognizant.ems.repository.EmployeeRepository;

@Service
public class EmployeeService {

    @Autowired
    private EmployeeRepository employeeRepository;
```

```

        public Page<Employee> getEmployees(Pageable pageable) {

            return employeeRepository.findAll(pageable);

        }
    }

package com.cognizant.ems.controller;

import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.data.domain.Page;
import org.springframework.data.domain.PageRequest;
import org.springframework.data.domain.Pageable;
import org.springframework.data.domain.Sort;
import org.springframework.web.bind.annotation.*;

import com.cognizant.ems.entity.Employee;
import com.cognizant.ems.service.EmployeeService;

@RestController
@RequestMapping("/employees")

public class EmployeeController {

    @Autowired
    private EmployeeService employeeService;

    @GetMapping
    public Page<Employee> getEmployees(

        @RequestParam(defaultValue = "0") int page,

        @RequestParam(defaultValue = "5") int size,

        @RequestParam(defaultValue = "id") String sortBy) {

        Pageable pageable = PageRequest.of(page, size, Sort.by(sortBy));

        return employeeService.getEmployees(pageable);}}

```

Exercise 7: Employee Management System - Enabling Entity Auditing

Business Scenario:

Implement auditing to track the creation and modification of employees and departments.

Instructions:

1. Entity Auditing:

- Enable auditing in your application by configuring auditing properties.
- Use annotations like **@CreatedBy**, **@LastModifiedBy**, **@CreatedDate**, and **@LastModifiedDate**.

```
package com.cognizant.ems.config;

import org.springframework.context.annotation.Configuration;
import org.springframework.data.jpa.repository.config.EnableJpaAuditing;

@Configuration
@EnableJpaAuditing
public class AuditConfig {

}

package com.cognizant.ems.entity;

import java.time.LocalDateTime;
import org.springframework.data.annotation.CreatedBy;
import org.springframework.data.annotation.CreatedDate;
import org.springframework.data.annotation.LastModifiedBy;
import org.springframework.data.annotation.LastModifiedDate;
import org.springframework.data.jpa.domain.support.AuditingEntityListener;
import jakarta.persistence.*;
import lombok.Data;

@Entity
@Table(name="employee")
```

```
@EntityListeners(AuditingEntityListener.class)

@Data

public class Employee {

    @Id

    @GeneratedValue(strategy=GenerationType.IDENTITY)

    private Long id;

    private String name;

    private String email;

    @ManyToOne

    @JoinColumn(name="department_id")

    private Department department;

    @CreatedBy

    private String createdBy;

    @LastModifiedBy

    private String lastModifiedBy;

    @CreatedDate

    private LocalDateTime createdDate;

    @LastModifiedDate

    private LocalDateTime lastModifiedDate;

}

package com.cognizant.ems.entity;

import java.time.LocalDateTime;

import java.util.List;

import org.springframework.data.annotation.CreatedBy;

import org.springframework.data.annotation.CreatedDate;
```

```
import org.springframework.data.annotation.LastModifiedBy;
import org.springframework.data.annotation.LastModifiedDate;
import org.springframework.data.jpa.domain.support.AuditingEntityListener;
import jakarta.persistence.*;
import lombok.Data;

@Entity
@Table(name="department")
@EntityListeners(AuditingEntityListener.class)
@Data
public class Department {

    @Id
    @GeneratedValue(strategy=GenerationType.IDENTITY)
    private Long id;

    private String name;

    @OneToMany(mappedBy="department", cascade=CascadeType.ALL)
    private List<Employee> employees;

    @CreatedBy
    private String createdBy;

    @LastModifiedBy
    private String lastModifiedBy;

    @CreatedDate
    private LocalDateTime createdDate;

    @LastModifiedDate
    private LocalDateTime lastModifiedDate;
}
```

```
package com.cognizant.ems.config;

import java.util.Optional;

import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.data.domain.AuditorAware;

@Configuration

public class AuditorAwareConfig {

    @Bean

    public AuditorAware<String> auditorProvider() {

        return () -> Optional.of("Admin");

    }

}
```

Exercise 8: Employee Management System - Creating Projections

Business Scenario:

Create projections to fetch specific data subsets from the employee and department entities.

Instructions:

1. Projections:

- Define interface-based and class-based projections.
- Use **@Value** and constructor expressions to control the fetched data.

```
package com.cognizant.ems.projection;

public interface EmployeeProjection {

    Long getId();

    String getName();

    String getEmail();}

package com.cognizant.ems.dto;
```



```
public class EmployeeDTO {  
    private Long id;  
    private String name;  
    private String email;  
    public EmployeeDTO(Long id, String name, String email) {  
        this.id = id;  
        this.name = name;  
        this.email = email;  
    }  
    public Long getId() {  
        return id;  
    }  
    public String getName() {  
        return name;  
    }  
    public String getEmail() {  
        return email;  
    }  
}
```

